

Лабораторная работа №2

Структуры данных в Julia

Ведьмина Александра Сергеевна

2026-09-20

- Цель и инструменты
- Основные структуры данных
- Операции над множествами
- Построение массивов
- Случайные векторы
- Простые числа и суммы
- Воспроизводимость
- Результаты и выводы

Section 1

Информация

- Ведьмина Александра Сергеевна
- студентка
- кафедра теории вероятностей и кибербезопасности
- Российский университет дружбы народов
- 1132236003@rudn.ru



Section 2

Вводная часть

Цель: изучить структуры данных Julia и применить операции над ними для решения вычислительных задач.

Задачи:

- повторить примеры для кортежей, словарей, множеств и массивов;
- выполнить задания 1–6;
- обработать случайные векторы длины 250;
- применить пакет Primes;
- обеспечить воспроизводимость результатов;
- проверить решения автоматическими тестами.

- Julia и стандартные библиотеки Random, Statistics, Test;
- Primes.jl для вычисления простых чисел;
- Jupyter Lab и IJulia;
- DrWatson для структуры проекта;
- Literate.jl для производных материалов;
- Quarto для отчёта и презентации.

Section 3

Структуры данных

Кортежи и словари

```
x1 = (1, 2, 3)
x2 = (1, 2.0, "tmp")
x3 = (
    a=2,
    b=1 + 2,
)

x2[1]
x3.a
"tmp" in x2
```

```
phonebook = Dict(
    "Иванов И.И." => (
        "867-5309",
        "333-5544",
    ),
    "Бухгалтерия" =>
        "555-2368",
)
```

- Tuple неизменяем и индексируется с единицы;
- Dict хранит пары «ключ – значение».

Для множеств

$$A = \{0, 3, 4, 9\}, \quad B = \{1, 3, 4, 7\}, \quad C = \{0, 1, 2, 4, 7, 8, 9\}$$

вычислено объединение попарных пересечений:

```
P = union(  
    intersect(A, B),  
    intersect(A, C),  
    intersect(B, C),  
)
```

Результат: $P = \{0, 1, 3, 4, 7, 9\}$.

Результат операций над множествами

```
asvedjmina@veddy-msi:~/university/static_analys$ julia --project=labs/lab02/project labs/lab02/project/scripts/run_task.jl 1
  Activating project at `~/university/static_analys/labs/lab02/project`
A = Set{Int64}([0, 4, 9, 3])
B = Set{Int64}([4, 7, 3, 1])
C = Set{Int64}([0, 4, 7, 2, 9, 8, 1])
P = Set{Int64}([0, 1, 3, 4, 7, 9])
asvedjmina@veddy-msi:~/university/static_analys$
```

- повтор $A \cap B$ в условии не влияет на объединение;
- отдельно проверены множества элементов разных типов.

Section 4

Построение массивов

Диапазоны

При $N = 25$ построены три последовательности:

```
ascending = collect(1:N)
```

```
descending = collect(  
    N:-1:1,  
)
```

```
there_and_back = vcat(  
    1:N,  
    (N - 1):-1:1,  
)
```

```
asvedjmina@veddy-msi:~/university/static_analys$ julia --project=labs/lab02/project labs/lab02  
/project/scripts/run_task.jl 3.3  
Activating project at `~/university/static_analys/labs/lab02/project`  
3.3: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 2  
5, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

Повторение элементов

```
tmp = [4, 6, 3]

first_ten = fill(
    first(tmp),
    10,
)

counts_11_10_10 = vcat(
    first(tmp),
    repeat(tmp, 10),
)
```

- 4 встречается 11 раз;
- 6 и 3 встречаются по 10 раз.

Блочные повторения

```
tmp = [4, 6, 3]

blocks_10_20_30 = vcat(
    fill(tmp[1], 10),
    fill(tmp[2], 20),
    fill(tmp[3], 30),
)
```

```
asvedjmina@veddy-msi:~/university/static_analys$ julia --project=labs/lab02/project labs/lab02  
/project/scripts/run_task.jl 3.8  
    Activating project at `~/university/static_analys/labs/lab02/project`  
3.8: [4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6,  
      , 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]  
asvedjmina@veddy-msi:~/university/static_analys$
```

Степени двойки

```
powers = vcat(  
    2.^ tmp[1:2],  
    fill(  
        2^tmp[3],  
        4,  
    ),  
)  
  
six_digits = count(  
    ==('6'),  
    join(powers),  
)
```

```
asvedjnina@veddy-msi:~/university/static_analys$ julia --project=labs/lab02/project labs/lab02  
/project/scripts/run_task.jl 3.9  
Activating project at `~/university/static_analys/labs/lab02/project`  
3.9: [16, 64, 8, 8, 8, 8]  
Количество цифр 6: 2; элементов, равных 6: 0  
asvedjnina@veddy-msi:~/university/static_analys$
```

Результат: [16, 64, 8, 8, 8, 8].

Функциональные последовательности

В точках $x = 3, 3, 1, \dots, 6$ вычислено $y = e^x \cos x$:

```
x_grid = collect(  
    3.0:0.1:6.0,  
)  
y_grid = (  
    exp.(x_grid) .*  
    cos.(x_grid)  
)  
y_mean = mean(y_grid)
```

Среднее значение:

$$\bar{y} = 53,1137459464.$$

Section 5

Случайные векторы

Воспроизводимая выборка

```
rng = MersenneTwister(2026)
x = rand(rng, 0:999, 250)
y = rand(rng, 0:999, 250)
```

Фиксированный seed обеспечивает одинаковые выборки при каждом запуске.

Построены:

- разности $y_{i+1} - x_i$;
- линейные комбинации $x_i + 2x_{i+1} - x_{i+2}$;
- отношения $\sin(y_i) / \cos(x_{i+1})$;
- корни абсолютных отклонений от среднего.

```
differences = (  
    y[2:end] .-  
    x[1:end-1]  
)  
  
linear_combination = (  
    x[1:end-2] .+  
    2 .* x[2:end-1] .-  
    x[3:end]  
)  
  
trig_ratios = (  
    sin.(y[1:end-1]) ./  
    cos.(x[2:end])  
)
```

Отбор и сортировка

```
indices = findall(>(600), y)
y_over_600 = y[indices]
corresponding_x = x[indices]

x_sorted_by_y = x[
    sortperm(y)
]
x_top_10 = sort(
    x;
    rev=true,
)[1:10]
unique_x = unique(x)
```

- сохраняется соответствие позиций x и y ;
- сортировка x выполняется по значениям y .

Показатель	Значение
$y > 600$	104
Не далее 200 от $\max(y)$	49
Чётных / нечётных x	124 / 126
Кратных 7	32
Уникальных x	230

Топ-10:

995, 983, 980, 976, 975,
974, 971, 968, 966, 964

Section 6

Простые числа и суммы

Квадраты и простые числа

```
squares = [  
    i^2 for i in 1:100  
]  
  
myprimes = collect(  
    primes(1000)  
)  
  
prime_89 = myprimes[89]  
prime_slice = myprimes[  
    89:99  
]
```

- количество простых чисел — 168;
- 89-е простое число — 461;
- срез 89:99 — от 461 до 523.

Вычисление сумм

Выражение	Результат
$\sum_{i=1}^{100} (i^3 + 4i^2)$	26 852 735
$\sum_{i=1}^{25} (2^i/i + 3^i/i^2)$	$2,12917 \cdot 10^9$
Ряд произведений	6,9763461379

```
terms = vcat(  
    1.0,  
    accumulate(  
        *,  
        [2k / (2k + 1)  
        for k in 1:19],  
    ),  
)
```

Section 7

Воспроизводимость

Jupyter Notebook

```
[1]: using Pkg
Pkg.activate(joinpath(pwd(), ".."))
Pkg.instantiate()
using Lab02DataTypes, Primes, Random, Statistics

# Activating project at "~/university/static_analysis/labs/lab02/project"
# warning: The active manifest file has dependencies that were resolved with a different julia version (1.10.12). U
# nrespected behavior may occur.
# warning: The project dependencies or compat requirements have changed since the manifest was last resolved.
# It is recommended to 'Pkg.resolve()' or consider 'Pkg.update()' if necessary.
# 0 dependencies successfully precompiled in 0 seconds, 43 already precompiled.
```

2.2. Общие операции и кортежи

```
[2]: empty tuple = ()
favoriteLang = ("Python", "Julia", "R")
x1 = (1, 2, 3)
x2 = (1, 2, 0, "top")
x3 = (true, 10, -1)
(isempty(empty tuple), length(x2), x2[1:], x3[1:] + x1[1],
x3.a, x3.b, x3[2], in("top", x2), 0 in x2,
unique([1, 2, 1, 3]), reduce(+, x3), maximum(x3))
```

2.2.2. Словари

```
[3]: phonebook = Dict{String, String} => {"807-53001", "333-5544"},
"bryant@cs.cmu.edu" => "555-7384"

Would you like to get notified about official
Jupyter news?
[Open in new tab] Yes No
```

Начало notebook.

```
[1]: function summary(differences)
    multiples = 7 * rand(1000)
    x = sortby(rand(1000), by = y -> y % 7)
    x_top_10 = rand(1000)
    unique_x = unique(x)
end

summary([100, 211, -913, 132, -386, 45, 423, 236, 247, 119, -910, 873, -133, 256, 69, -263, 953, -5, 58,
281, linear combination = [206, 1783, 1024, 3032, 2264, 326, 633, 1284, 914, 3038, -384, -233, 2217, 636, 969, 22
27, 1513, 349, 1377, 1394], trig ratios = [-0.939917726528547, 1.116047624150875, 1.7564403134945025, -0.58311807
07187156, 3.534468626943689, -1.426613613414744, 0.572261466313218, -0.45171086477393436, 1.563651131124571, -0.
231181202612359, 1.926420929737823, -2.944646644686695, 0.6923292940734411, -0.273610223272387, 1.6250214499
18664, -1.036523292322825, 1.469851242817377, -0.6993527693987425, 0.975264812675362, -1.0182968585944664], un
lighted sum = 0.861337984761561851, y over 600 = [861, 774, 676, 458, 793, 773, 814, 829, 938, 678, -623, 955, 93
7, 636, 461, 858, 473, 829, 958, 816], over 400 indices = [1, 2, 4, 7, 9, 10, 12, 15, 17, 19, -232, 289, 242, 14
3, 244, 245, 246, 247, 249, 258], corresponding x = [877, 184, 379, 45, 526, 241, 199, 983, 565, 315, 808, 78, 7
49, 345, 789, 954, 470, 261, 788, 474], deviations = [15.57762735507887, 17.5994318098182, 22.254437759691866, 10.
718776782616759, 15.143334323221893, 27.29913238248697, 21.38344423261675, 9.8736155360932693, 9.87976872846317,
15.697798569741914, -20.730171244668675, 15.976638264377261, 12.195980950729387, 17.183111262956703, 21.4536781614
53735, 4.67237186631104, 16.617989398747666, 15.95645259876316, 17.154868277954666, 4.4429718783866131], near_max
count = 21, even count = 12, odd count = 126, multiples of 7 = 37, x sorted by y = [963, 21, 445, 403, 429, 206, 6
3, 192, 453, 343, -973, 462, 76, 429, 978, 864, 111, 64, 754, 761, x_top_10 = [999, 995, 989, 989, 987, 988, 983,
981, 978, 973], unique_x = [877, 184, 379, 379, 723, 733, 45, 557, 526, 241, -663, 451, 345, 789, 954, 470, 381,
892, 788, 474])
```

2.4. Задания 4-6

```
[10]: sequences = squares and primes()
sums = sums_task(m=25)
(squares=sequences.squares, first 100 primes=sequences.primes,
prime 88=sequences.prime 88, primes 89 to 90=sequences.primes 89 to 90,
sum 6 1=sequences.sum 6 1, sum 6 2=sequences.sum 6 2, sum 6 3=sequences.sum 6 3)

[18]: (squares = [1, 4, 9, 16, 25, 36, 49, 64, 81, 100, -8281, 8404, 8649, 8816, 9025, 9216, 9409, 9604, 9801, 10000],
first 168 primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, -937, 941, 947, 953, 967, 971, 977, 983, 991, 997], prime
89 = 461, primes 89 to 99 = [461, 463, 467, 479, 487, 491, 493, 503, 509, 521, 523, 541, 547, 557, 563, 569, 577, 587, 593, 599, 601, 607, 611, 613, 617, 619, 623, 629, 631, 637, 641, 643, 647, 653, 659, 661, 667, 671, 673, 677, 683, 687, 691, 697, 701, 703, 709, 713, 719, 727, 731, 733, 739, 743, 749, 757, 761, 769, 773, 779, 787, 791, 797, 803, 809, 811, 817, 821, 823, 827, 829, 833, 839, 843, 847, 851, 853, 857, 859, 863, 867, 877, 881, 883, 887, 893, 899, 907, 911, 913, 917, 919, 923, 929, 931, 937, 941, 943, 947, 953, 959, 967, 971, 973, 977, 983, 989, 991, 997], prime
sum 6 1 = 28852735, sum 6 2 =
2.1291791365143462e3, sum 6 3 = 6.37634613798161e3)
```

Результаты заданий.

Из единого файла `scripts/lab02_literate.jl` получены:

- `generated/lab02_code.jl`;
- выполненный `lab02_notebook.ipynb`;
- `lab02_quarto.qmd`;
- `lab02_quarto.html`.

```
project=labs/lab02/project
julia --project=$project \
    $project/scripts/generate_outputs.jl
```

Автоматические тесты

```
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab02/project/notebooks$ cd /home/asvedjmina/university/static_analys
julia --project=labs/lab02/project \
  labs/lab02/project/test/runtests.jl
Activating project at `~/university/static_analys/labs/lab02/project`
Test Summary: | Pass Total Time
Множества | 2 2 0.6s
Test Summary: | Pass Total Time
Массивы | 10 10 0.7s
Test Summary: | Pass Total Time
Случайные векторы | 6 6 0.9s
Test Summary: | Pass Total Time
Квадраты, простые числа и суммы | 5 5 0.5s
(jupyter) asvedjmina@veddy-msi:~/university/static_analys$
```

Проверены:

- множества — 2 теста;
- массивы — 10 тестов;
- случайные векторы — 6 тестов;
- числа и суммы — 5 тестов;
- структура и материалы — 9 тестов.

Итого: 32 из 32 проверок пройдены.

Section 8

Заключение

- выполнены все примеры и самостоятельные задания;
- освоены кортежи, словари, множества и массивы;
- применены срезы, broadcasting, фильтрация и сортировка;
- обработаны воспроизводимые случайные выборки;
- найдены простые числа и вычислены три суммы;
- создан воспроизводимый `literate`-конвейер;
- все автоматические тесты пройдены.

Цель работы достигнута.

Структуры данных Julia позволяют компактно выражать операции над коллекциями, а broadcasting и индексирование дают возможность выполнять векторные вычисления без явных циклов.

Фиксированное окружение, seed генератора и единый literate-источник делают полученные результаты полностью воспроизводимыми.